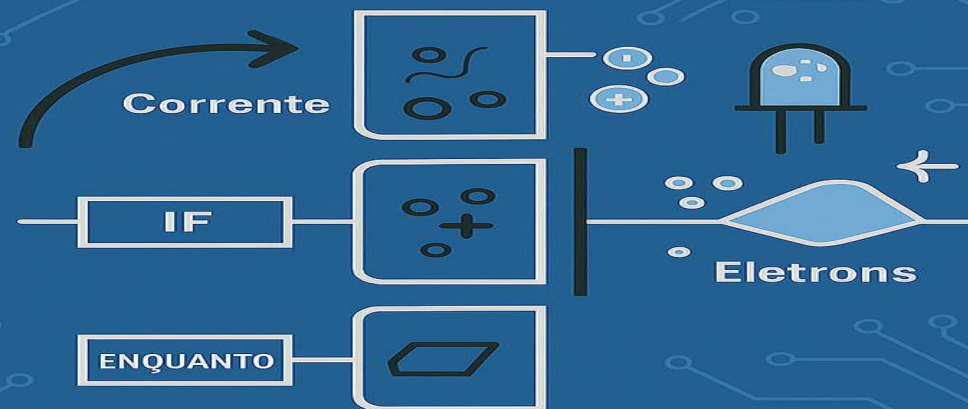


LÓGICA DE PROGRAMAÇÃO

TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS



HILTON ELIAS TOMASZEZISIIC DOS SANTOS
PEDRO GRAÇAS ALVES JUNIOR

SENAI

Desenvolvi esta Aula abrangente, repleta de exercícios para oferecer amplas oportunidades de prática. Quase todas as aulas incluem uma correspondente sessão de exercícios, proporcionando a chance de resolver problemas juntamente comigo.

Solicito sua dedicação e esforço para que, ao término desta unidade curricular, você esteja preparado para iniciar os estudos em uma linguagem de programação.

Este estudo abrangerá todos os fundamentos necessários para dar o primeiro passo como desenvolvedor.

Além de ser seu instrutor, pretendo também ser seu motivador, pois aprender programação é desafiador, especialmente nos primeiros estágios.

Nesta aula, iniciaremos o plano de estudos, com os conceitos fundamentais de algoritmos, lógica de programação e a linguagem que utilizaremos, será o Portugol.

Iremos iniciar através do cronograma da unidade curricular. Nesta aula em particular, abordaremos a introdução, variáveis, tipos primitivos e operações de entrada e saída de dados.

A seguir, abordaremos os operadores, incluindo os aritméticos, relacionais e lógicos. Em relação às estruturas condicionais, dedicaremos varias aulas para explorar a utilização da estrutura **SE simples e composta**.

Após isso, teremos sessões de exercícios.

Abordaremos a estrutura chamada "**Escolha Caso**" e apresentaremos alguns exercícios para prática, discutiremos as **estruturas de repetição**, que compreendem as **estruturas enquanto, repita e para**.

Exploraremos os **conceitos de procedimentos e funções**, destacando suas diferenças, os tópicos de **variável global** e **variável local**, explicando como são utilizados, discutiremos a passagem de valores por referência em procedimentos e funções.

Vetores e matrizes também serão temas abordados, acompanhados por uma série de exercícios práticos.

As aulas serão centradas na aplicação prática, proporcionando conceitos seguidos imediatamente por exercícios.

Finalmente, concluiremos com a criação de um algoritmo para algum jogo como o último exercício.

LÓGICA DE PROGRAMAÇÃO

A primeira coisa que nós temos que entender é o que é um algoritmo. Para começar a entender algoritmos, temos que pensar no dia a dia.

O que nós normalmente fazemos quando nós acordamos?

Nós Levantamos, depois tomamos banho, trocamos de roupa, tomamos o café da manhã, pegamos o ônibus ou de carro, desce do ônibus ou do carro.... Então, essa sequência de passos que eu estou mostrando aqui nada mais é que um algoritmo.

O algoritmo vai descrever como uma coisa acontece.

Vamos ver outro exemplo:

Eu estou em uma faixa de pedestre e quero atravessar a rua.

O que eu faço?

Primeiro vou até a faixa, olho para a direita, olho para a esquerda e aí sim eu atravesso. Se eu esquecer, por exemplo, de olhar para a direita ou olhar para a esquerda, isso pode ocorrer um acidente. Um carro pode vir correndo e me atropelar e eu nem vi isso acontecer.

Por que eu estou falando isso?

Algoritmos para programação é a mesma coisa.

Se você esquece de um passo, no seu código, no seu algoritmo, pode gerar um erro no seu programa. Então, temos que prestar bastante atenção, porque o computador não pensa. Embora nós temos hoje máquinas muito potentes, elas não pensam, elas já são programadas passo a passo para fazer o que elas fazem. Então, você precisa ensinar o computador ao que ele tem que fazer. O computador não pensa, então nós, que pensamos, vamos mostrar para ele passo a passo o que é para ele fazer.

Isso é algoritmo. Apesar de hoje em dia já termos a inteligência artificial, eu acredito que nunca chegaremos ao ponto em que um computador será capaz de pensar sozinho, sem que alguém o programe para isso.

O programe para fazer um cálculo ou qualquer coisa, qualquer situação, é passo a passo. Por isso, você precisa aprender algoritmos para você ser um futuro desenvolvedor.

O computador tem uma linguagem, tem uma maneira de entender o que a gente quer passar.

Então, nós temos que aprender a programar da maneira que ele entende. Hoje em dia, tudo envolve programação.

- ✓ O sistema para um avião envolve programação.
- ✓ O celular que você usa envolve programação.
- ✓ Até geladeiras hoje já temos com inteligência artificial, com uma programação pesada.
- ✓ Temos óculos 3D.
- ✓ O Google Play que você usa aí no celular tem programação.
- ✓ E não vai parar por aí, não.

No mundo que nós vivemos, tão moderno, aprender a programar é investir no futuro, porque é uma área não para de crescer. A área da tecnologia, a área de TI, é muito gigantesca e você tem aí diversas oportunidades ao redor do mundo que envolvem a programação.

Eu dei um exemplo do dia a dia, agora eu vou mostrar um exemplo de somar dois números.

Como nós faríamos para somar dois números?

Eu quero criar um programa, bem simples, que só vai somar dois números.

Vamos começar com o básico.

Como vou somar dois números, preciso desses números, preciso que a pessoa digite esses números.

Então, nós vamos pedir o número do teclado.

1. Primeiro passo é pedir o número do teclado.
2. Em seguida, nós pedimos mais um número, porque para somar, nós precisamos pelo menos de dois números.
3. E nós criaremos um algoritmo para calcular esses números.
4. Calculamos, próximo passo, mostrar o resultado na tela.

Então, algoritmos é mais ou menos assim.

Calcular uma idade.

Pense aí como você calcularia uma idade.

Como você faria um programa para calcular a idade?

1. Primeira coisa, posso pedir o ano de nascimento dessa pessoa via teclado. Peguei essa idade.

Como é calculado a idade de uma pessoa?

Pegamos o ano atual e subtraí com o ano de nascimento dessa pessoa.

2. O próximo passo é pegar o ano atual.

Como nós fazemos isso?

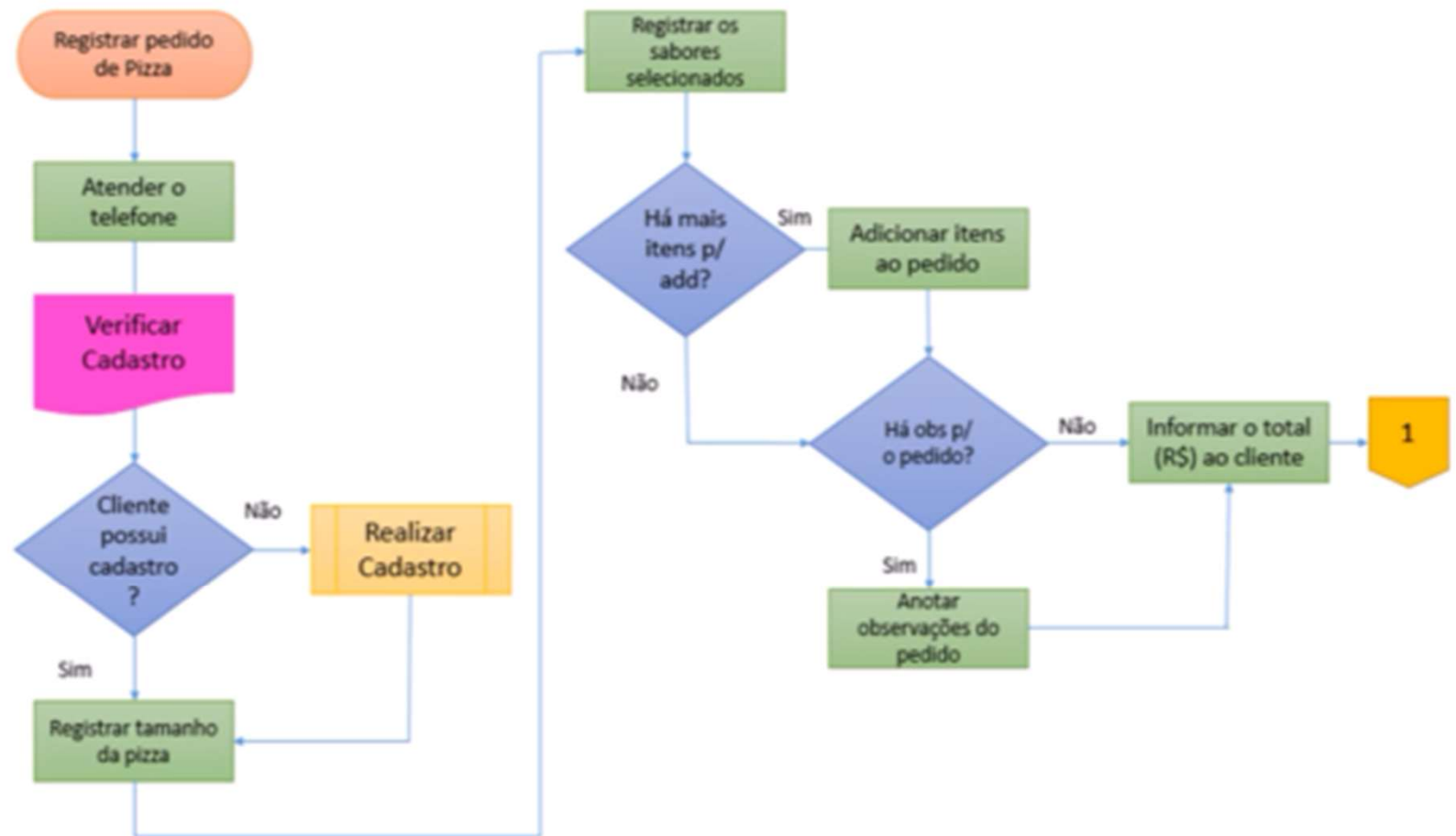
Podemos pegar no sistema.

2. O próximo passo é pegar o ano de nascimento da pessoa.
3. Pegamos o ano atual e a data de nascimento dessa pessoa, nós vamos fazer o cálculo.
4. Então, subtraí o ano atual com o ano de nascimento e mostra na tela.

Esse é mais um exemplo de algoritmo.

FERRAMENTAS

Fluxograma



FERRAMENTAS

Antigamente tínhamos o **fluxograma**, que é um passo a passo também, só que em forma de desenho. Isso já não é muito utilizado mais. Pode ser que planejem o programa antes de colocar a mão na massa, só que eu não vejo mais a utilização dele.

Já o **Portugol** é muito mais parecido com **linguagem de programação**, só que ele também é baseado nesse **fluxograma**. Por exemplo, aqui veremos algumas figuras, onde temos **entradas, saídas e comandos de decisão**.

Se isso for **verdade**, se alguma coisa for verdade, por exemplo:

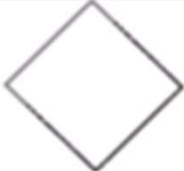
Cliente, possui cadastro?

Se sim, eu faço uma coisa;

Se não, eu vou para outro caminho e faço uma outra coisa.

Essa é uma decisão que o programa vai tomar. E tem vários símbolos.

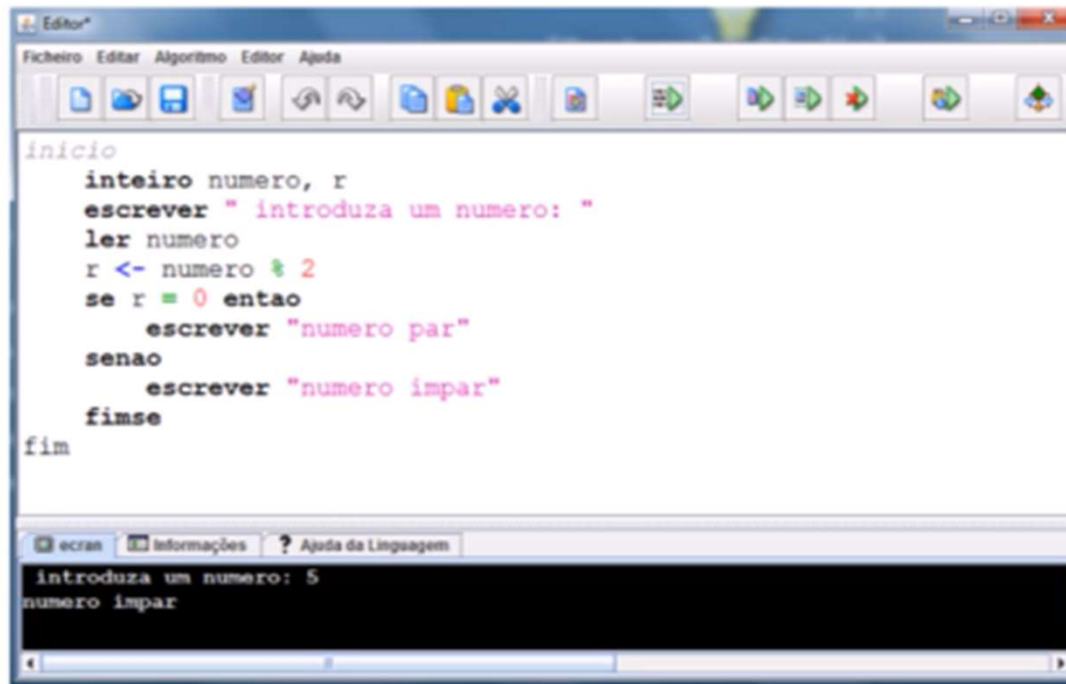
Temos o quadrado, o círculo inicial.... Cada coisa serve para alguma utilidade. Vejam um gráfico aqui com a figura e o seu significado:

FIGURA	SIGNIFICADO
	Figura para definir início e fim do algoritmo
	Figura usada no processamento de cálculo, atribuições e processamento de dados em geral
	Figura utilizada na representação de entrada de dados
	Figura utilizada para representação da saída de dados
	Figura que indica o processo seletivo ou condicional, possibilitando o desvio no caminho do processamento
	Símbolo geométrico usado como conector
	Símbolo que identifica o sentido do fluxo de dados, permitindo a conexão entre as outras figuras existentes

Nós iremos utilizar o **Portugol**, ou também chamado de **pseudocódigo**. Ele é o mais utilizado atualmente, e é mais próximo da **linguagem de programação**.

Então veja um exemplo, que faz a verificação se o número é par ou ímpar e mostrar na tela.

Tem várias ferramentas para você aprender **pseudocódigo**. A que nós utilizaremos é o **Visualg**, que é muito conhecido.



The image shows a screenshot of the Visualg IDE. The main window displays a Portugol program with the following code:

```
inicio
    inteiro numero, r
    escrever " introduza um numero: "
    ler numero
    r <- numero % 2
    se r = 0 entao
        escrever "numero par"
    senao
        escrever "numero impar"
    fimse
fim
```

At the bottom, there is a console window titled "ecran" showing the output of the program:

```
introduza um numero: 5
numero impar
```

Agora vou apresentar pra vocês a diferença de algo muito importante. Quando você aprende algoritmos, você aprende o funcionamento de uma **linguagem estrutural**. Como assim? Existe a **linguagem estrutural** e a **linguagem orientada a objetos**.

A **linguagem estrutural** é aquela que segue um padrão, digamos assim, um passo a passo. Terminou de executar a primeira linha, vai para a segunda, terminou de executar a segunda, vai para a terceira e assim sucessivamente.

Não que a **linguagem orientada a objetos** não seja assim, a **linguagem orientada a objetos** também é assim, só que ela é muito mais **dinâmica**. Ela tem **classes**, uma classe chama a outra, comunica com a outra. Mas é muito importante, você primeiro aprender a **linguagem estrutural** e só depois aprender a **linguagem orientada a objetos**.

Se você der um passo maior que a perna, digamos assim, tentar ir direto aprendendo uma **linguagem orientada a objetos**, vai chegar um ponto que você irá dizer: não estou entendendo nada, vou parar.

Então, é melhor você ir devagar. É por isso que o **Portugol** não é **orientado a objeto**, ele é **estrutural** e é isso que nós aprenderemos.

Para vocês conhecerem que **linguagens são estruturais**, temos, por exemplo:

A **linguagem C** é um tipo de **linguagem** que é **estrutural**.

Então, como falei, você tem, por exemplo, seis comandos, serão executados linha após linha de comando.

- Primeiro será o comando 1;
- Depois será o comando 2;
- Depois será o comando 3;
- Depois será o comando 4;
- Depois será o comando
- e assim por diante.

C

comando 1
comando 2
comando 3
comando 4
comando 5
comando 6

Diferente, não tão **diferente**, mas é **diferente** de uma **linguagem orientada a objetos** como, por exemplo, **C++**, **C#**, **Java** e etc.

Hoje em dia, o que é mais utilizado?

A linguagem orientada a objetos.

A linguagem orientada a objeto é o que domina o mercado.

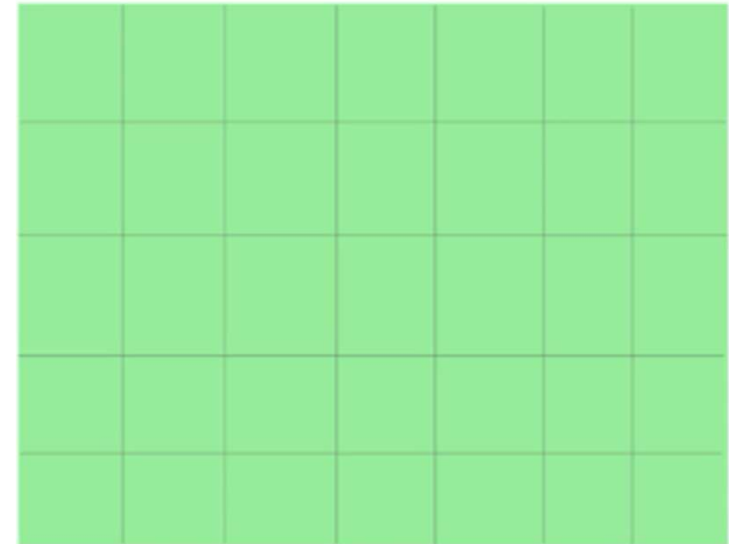
Algumas pessoas e empresas ainda utilizam a **linguagem C**, porém, isso logo vai acabar e todo mundo vai utilizar a **linguagem orientada a objeto**, que é muito mais fácil de utilizar, muito mais avançado e interessante.

VARIÁVEIS

É fundamental nós entendermos como é o funcionamento do computador em relação às variáveis.

Bom, para entendermos variáveis, temos que imaginar um armário, onde tem várias gavetas, ou seja, vários lugares para que você armazene alguma coisa.

Então, imagine que cada gaveta dessa vai armazenar um tipo de dado diferente.



Como assim?

O nosso computador tem memória, afinal de contas ele guarda informações, se não tiver memória, não é possível nem sequer você executar um programa, porque precisa da memória volátil.

E da mesma forma, para escrever algoritmos, nós precisaremos utilizar dessa memória do computador para armazenar algumas informações, que nós chamamos de variáveis.

Então, existem vários tipos de variáveis.

Para exemplificar, vamos imaginar assim:

Nesse primeiro quadrado, eu posso guardar apenas os alimentos.

Então, eu não posso guardar objetos ou outras coisas, apenas alimentos.

Lá no último quadrado, eu posso guardar apenas objetos, ou seja, apenas esse tipo de informação que é objetos, entre outros.

Então, se eu quero guardar alimentos, naquele primeiro quadrado, vamos guardar uma informação do tipo alimento, que será frutas.

Frutas é um alimento, então eu posso guardar o que mais?

Verduras. As verduras também são um alimento que eu posso guardar, ou seja, se eu tentar armazenar um objeto nesse quadrado que só armazena alimentos, então nós teremos um erro lá no nosso programa.

Da mesma forma, objetos, eu vou guardar um guarda-chuva, eu vou guardar um sofá. Cada tipo de informação precisa guardar apenas aquele tipo de informação.

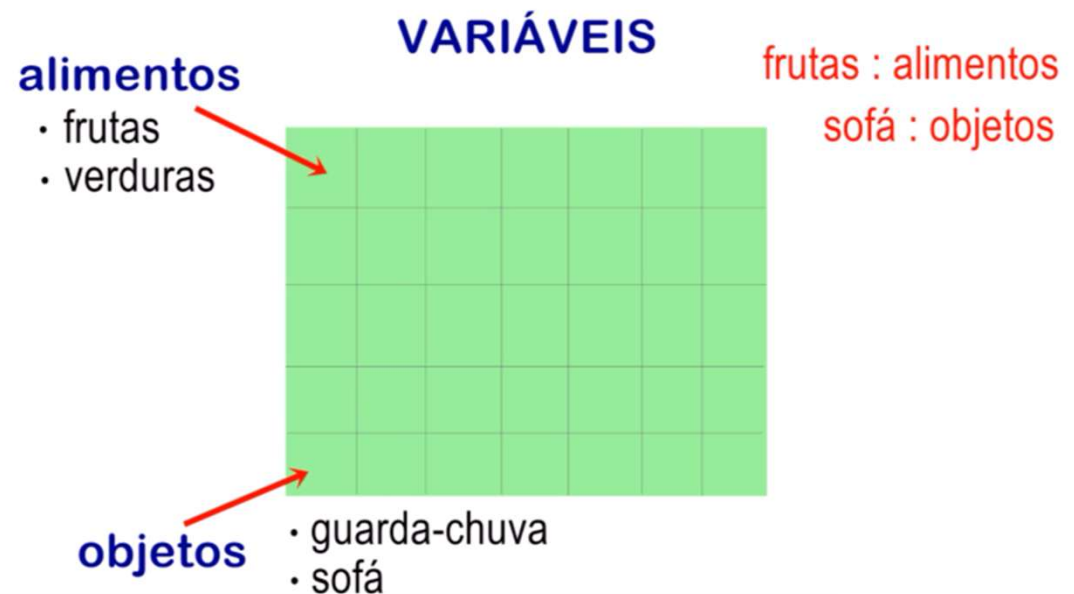
Como nós declaramos esses tipos de informações?

Vamos supor que esses alimentos, a palavra **alimentos** é o meu **tipo**, e **frutas** e **verduras** é o **nome do tipo** que eu estou dando.

Para declarar essa informação, o que eu vou falar?

Eu vou dar um nome, no meu caso eu dei **frutas**, dois **pontos**, e eu coloco o tipo de informação, que é **alimento**.

Então, por exemplo, **sofá**, dois **pontos**, **objetos**, ou seja, **sofá** é do **tipo objeto**, **fruta** é do **tipo alimento**.



Vamos ver agora quais são os tipos que nós temos em lógica de programação.
Nós temos o tipo:

- ✓ inteiro,
- ✓ Real
- ✓ Carácter
- ✓ lógico.

Quando você iniciar uma linguagem específica de programação, verá que existem mais tipos.

Por isso que o **Visualg**, **Portugol**, os algoritmos, quando nós aprendemos, temos que ter domínio sobre esses quatro, porque esses quatro são a base. Depois virão outros, mas que são baseados nesses quatro, que são de tamanhos menores ou maiores, você entenderá quando estiver estudando uma determinada **linguagem de programação**.

Os **inteiros**, como na matemática nós já aprendemos, eles são aqueles valores sem vírgula.

Por exemplo, o número **35**, o número **-1**, também é um **inteiro**, ou seja, um **inteiro negativo**, o número **0** também, tudo aquilo que não tem vírgula, aquilo que é **inteiro**, literalmente.

Os números **reais**, eles são aqueles números com vírgula.

12,566, **0,1** ou **-7,215**, tudo aquilo que envolve vírgula.

E você pode também considerar o número **inteiro** como **real**, afinal, na matemática, por exemplo, aquele número **35**, ele tem uma vírgula, ele está escondido, essa vírgula é depois do 5, sendo escrito assim **35,0**.

Mas, se você for realmente usar, por exemplo, valores de moeda **Reais**, de **Dólar**, que envolvem vírgula, então, pode utilizar **real**, que não vai ter problema durante a compilação do seu programa.

Nós também temos o tipo **carácter**, que guarda palavras, e sempre que for utilizar o tipo **carácter**, sempre colocar essa palavra em **aspas**. Se você colocar números entre **aspas**, ele não vai ser considerado como **inteiro** ou **real**, ele será considerado como **carácter**.

Veremos que teremos como converter esse número que está armazenado em **carácter** para **inteiro** ou para **real** também.

E o valor **lógico**, que vai receber **verdadeiro** ou **falso**.

Como declarar essas variáveis?

Já vai se acostumando com essas palavras, **declarar**, **atribuir**, que vai ser utilizado muito.

Declarar é definir uma variável que eu vou utilizar.

nome : caractere

Por exemplo, eu quero guardar o nome de uma pessoa, vou chamar essa variável de **nome**, eu coloco **dois pontos** e o **tipo**.

idade : inteiro

Nome são letras, palavras, então, colocarei o tipo **carácter**.

dolar : real

E outra variável, do tipo **inteiro**, é para armazenar a idade, que eu chamei de **idade**.

tem : logico

Dólar, do tipo **real**, afinal, tem vírgulas.

E uma variável do tipo **lógico**, chamado **tem**. Ela vai me dizer se **tem** ou **não**, **verdadeiro** ou **falso**.

O que você não pode fazer quando declarar uma variável?

Por exemplo, coloquei aqui 2nomes. Uma variável sempre começará com **carácter**, não **números**, essa forma está errada.

~~2nome : caractere~~

Outro erro é você colocar **espaço**. Se eu quero escrever, dar o nome da minha variável de **nome completo** e eu dou espaço ela estará errada.

~~nome completo : caractere~~

Variáveis sempre são palavras grudadas, então, mesmo que eu quisesse escrever **nome completo**, ali o completo, o **C**, começará com letra maiúscula para a gente poder separar isso. Então, essa forma também está errada. Aqui seria a maneira correta, eu quero escrever duas palavras, nome completo.

✓ nomeCompleto : caractere

Aqui também está errado, porque variáveis não podem ter acentuação. Outra forma também, símbolos, eu sempre tenho que começar com carácter.

~~dólar : real~~

~~\$dolar : real~~

Essa forma é um jeito que algumas pessoas utilizam. É mais utilizado você colocar o C em maiúscula, mas muitas pessoas utilizam essa, mas não está errado.

nome_completo : caractere

Número1, você pode, ou seja, a partir da primeira palavra, então, utilizar números no meio das frases, essa forma é uma forma correta.

✓ numero1 : inteiro

E as palavras reservadas, por exemplo, **algoritmo**.

Por que eu não posso utilizar **algoritmo**?

Você verá que lá no programa já existe essa palavra, **algoritmo**, que é utilizada, então, é uma palavra reservada. Tudo que já estiver lá como palavra, que eu consiga ver, que eu saiba que tem, eu não posso utilizar.

~~algoritmo : caractere~~

Por exemplo, **carácter**, é uma palavra que é reservada, não posso chamar **carácter**, **dois pontos**, **carácter**. Ou seja, tenho que utilizar apenas palavras não existentes.

Atribuição significa você preencher uma variável. Quando nós declaramos, o nome do tipo **carácter**, no **Visualg**, elas são inicializadas vazias.

Por exemplo, **carácter** vem vazio, sem nenhuma palavra, **Idade** vem com zero. Então, nós temos que preencher essas variáveis para a gente poder utilizar.

Em uma **linguagem de programação**, normalmente, na maioria delas, assim que eu declaro, por exemplo, **nome** do tipo **carácter**, eles **não vêm vazio**. Vêm com **memória**, com **lixo de memória**, e aí precisamos **inicializar** elas. Por exemplo, **idade**, que é do tipo **inteiro**, vem com **lixo de memória**.

Inicializamos com zero, fazemos essa **atribuição**, colocando zero. Só que aqui no **Visualg**, não precisamos, porque ele já vem com zero.

Como fazer uma **atribuição**?

É como se fosse uma setinha. Você coloca símbolo de menor < e um tracinho -, e assim estará **atribuindo** aquele valor.

O **nome** é do tipo **carácter**.

Irei **atribuir** a palavra **Joana** para esse **nome**, para essa variável.

Idade, a mesma coisa, só que sem as **aspas**. As **aspas** são apenas para os **caracteres**.

Dólar recebe o **número com vírgula**.

E **tem** recebe **falso** ou **verdadeiro**.

atribuição

nome <- "joana"

idade <- 32

dolar <- 30,2654

tem <- falso

O que é errado fazer?

Atribuir um tipo errado. Exemplo: **nome** recebe **10**.

nome ← 10 ~~nome ← 10~~

Isso irá gerar um erro, e não queremos erro durante a compilação do nosso programa. Então, se você realmente quer adicionar o número **10** ao **nome** por alguma razão, você tem que colocar esse **número** entre **aspas**, para ser considerado como **caractere**.

ENTRADA E SAÍDA DE DADOS

Como é uma **saída de dados**?

Tenho um **nome** do tipo **carácter**, onde **atribuí Maria** a esse **nome**.

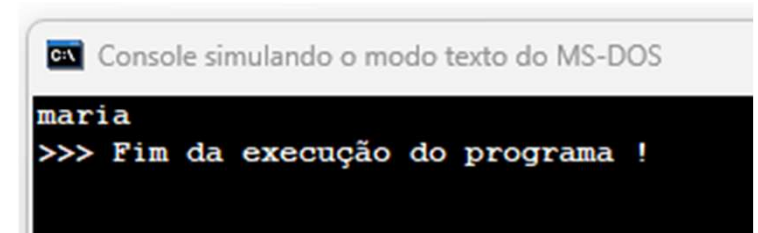
No **Visualg**, eu escrever a palavra, **escreva**, entre parênteses(), e colocar o nome dessa variável, que no caso é **nome**, e eu mandar executar o código, clicando no menu **Run(executar)**, ou pressionar **F9**, no teclado, o que acontecerá?

Aparecerá um console, uma Janela preta, e aparecerá a palavra **Maria**, que é o que está armazenado em **nome**.

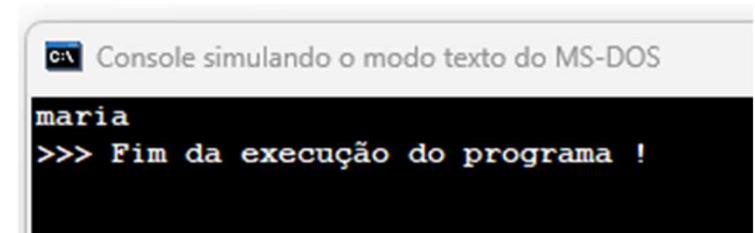
Então, não aparecerá **nome**, mas sim o que está armazenado em **nome**.

Posso colocar uma palavra qualquer entre **aspas**. temos duas formas de realizar uma saída. Você pode colocar variável ou colocar uma palavra.

```
1 Algoritmo "EscrevaNome"
2
3 Var
4 // Seção de Declarações d
5
6 nome:caractere
7
8 Inicio
9 // Seção de Comandos, pro
10
11 nome<-"maria"
12 escreva(nome)
13 Fimalgoritmo
```



```
1 Algoritmo "EscrevaNome"
2
3 Var
4 // Seção de Declarações d
5
6 nome:caractere
7
8 Inicio
9 // Seção de Comandos, pro
10
11 escreva("maria")
12 Fimalgoritmo
```



A mesma coisa para os outros tipos. Se eu tenho uma **idade** do tipo **inteiro**, onde eu atribui o valor **50**.

Então, escreva a **idade**, aparecerá o valor **50** na tela.

```
1 Algoritmo "EscrevaIdade"
2
3 Var
4 // Seção de Declarações da
5
6 idade: inteiro
7
8 Inicio
9 // Seção de Comandos, proce
10 idade <- 50
11 escreva(idade)
12 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

```
50
>>> Fim da execução do programa !
```

Ou posso também, escrever e colocar entre **aspas** o **número** que eu quero.

```
1 Algoritmo "EscrevaNumero"
2
3 Var
4 // Seção de Declarações das
5
6 Inicio
7 // Seção de Comandos, proce
8
9 escreva("123")
10 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

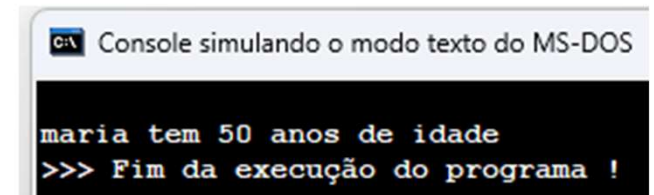
```
123
>>> Fim da execução do programa !
```

Como eu faço para escrever essa frase?

Maria tem 50 anos de idade. Ou seja, quero unir as duas variáveis em uma frase.

Escrevo abre parênteses, a variável **nome** vírgula - essa vírgula unirá todas as frases - vírgula **tem** entre **aspas** vírgula **idade** vírgula entre **aspas** **anos de idade** e fecha o parênteses.

```
1 Algoritmo "EscrevaFrase"
2
3 Var
4 // Seção de Declarações das variáveis
5
6 nome:caractere
7 idade:inteiro
8
9 Inicio
10 // Seção de Comandos, procedimento, funções,
11 nome<- "maria"
12 idade<- 50
13 Escreva(nome," tem",idade, " anos de idade")
14 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

```
maria tem 50 anos de idade
>>> Fim da execução do programa !
```

Algumas linguagens de programação, No Visual, também aceita o símbolo de + ao invés da vírgula, também vai fazer essa união de palavras.

Isso se chama **concatenação**. Quando você ouvir a palavra **concatene** ou **concatenar**, ou seja, é a mesma coisa que **juntar palavras**, perceba que **idade é do tipo inteiro**, e mesmo assim, ele se uniu a outras palavras do tipo **carácter**, porque ele é convertido automaticamente pelo visual, ele é considerado como **texto**.

ENTRADA E SAÍDA DE DADOS

Agora veremos a entrada de dados.

O que seria a entrada?

Seria pedir para uma pessoa digitar no teclado uma informação, que vai direto ser guardada dentro de uma variável.

Eu criei aqui, **nome** do tipo **carácter**, **idade** do tipo **inteiro**. E vou mostrar na tela. Escreva, irei mostrar na tela qual é o seu **nome**, então vai aparecer lá qual é o seu **nome**.

Na próxima linha escrevo, **leia**. A palavra **leia** é uma palavra **reservada**, onde vai guardar a informação que for digitar dentro da **variável** que eu colocar em **parênteses**, se eu coloco **leia, nome**, a palavra que a pessoa **digitar** e der Enter, essa **palavra vai ser guardada lá dentro**.

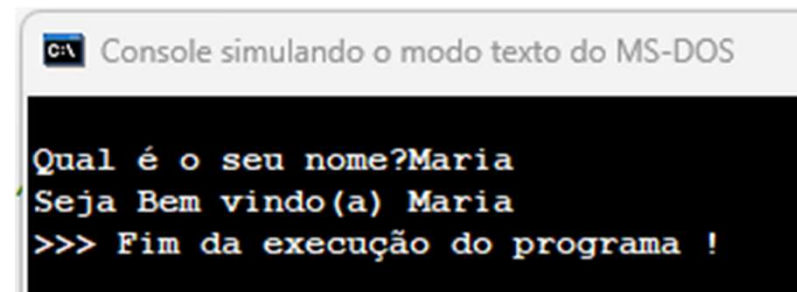
A seguir, posso digitar, **escreva**, seja bem-vindo, coloco uma **vírgula**, para se juntar com essa palavra que a pessoa digitou. Então, **seja bem-vindo, Maria**.

Isso é uma entrada de dados, a partir do momento que eu utilizo o **leia**.

Vamos para o Visualg, digite **escreva**, entre parênteses digite entre aspas duplas **"Seja bem-vindo(a)"**, coloque uma vírgula, para se juntar com essa palavra que a pessoa digitou: **Seja bem-vindo(a), Maria**. Isso seria uma entrada de dados, a partir do momento que eu utilizo o **leia**.

Onde temos **//** são comentários, eles servem para referenciar seu código, para que saiba o que está acontecendo, ele não interfere na programação.

```
1 Algoritmo "BemVindo"
2 // Descrição : Aqui você descreve o q
3
4 Var
5 // Seção de Declarações das variáveis
6
7 nome:caractere
8 idade:inteiro
9
10 Inicio
11 // Seção de Comandos, procedimento, fun
12
13 escreva("Qual é o seu nome?")
14 leia(nome)
15 escreva("Seja Bem vindo(a) ",nome)
16
17 Fimalgoritmo
```



```
C:\ Console simulando o modo texto do MS-DOS

Qual é o seu nome?Maria
Seja Bem vindo(a) Maria
>>> Fim da execução do programa !
```


OPERADORES

Veremos os operadores aritméticos, as **funções aritméticas**, a **ordem de precedência**, os **operadores relacionais** e os **operadores lógicos**.

Os **operadores aritméticos**, o nome até assusta, mas são aqueles **operadores matemáticos** que você conheceu no fundamental.

Vamos aprender como utiliza-los aqui.

Temos a **adição**, **subtração**, **multiplicação**, **divisão**, **divisão inteira**, **exponenciação**.

Os símbolos utilizados dentro do Visualg,

Operadores Aritméticos		
Adição	+	valor + valor
Subtração	-	valor – valor
Divisão	/	valor / valor
Multiplicação	*	valor * valor

A **divisão inteira**, nós colocamos uma barra invertida,

Operadores Aritméticos		
Divisão Inteira	\	valor \ valor

O que seria **divisão inteira**?

Quando realizamos a divisão, um número por outro, ele pode gerar um número com vírgula. Caso utilize a barra invertida, esse número com vírgula será desconsiderado, a parte da vírgula, ele retornará apenas a parte inteira. Pode ser que precise, em algum momento, em seu código.

Lembre-se, sempre quando você for realizar uma **divisão**, você precisa, **necessariamente**, utilizar uma variável do tipo **real**. Você não consegue **dividir** uma variável do tipo **inteira**, **duas** variáveis do tipo **inteira**, porque pode dar **erro** ou trazer um **resultado errado**. Ele pode pegar apenas o resultado inteiro e trazer para você como resposta. Então, fique de olho nisso.

A **potenciação** ou **exponenciação** é a operação matemática que representa a **multiplicação** de **fatores iguais**. Ou seja, usamos a **potenciação** quando um número é multiplicado por ele mesmo várias vezes.

Operadores Aritméticos		
Exponenciação	^	valor ^ valor

E o modulo retorna o resto da divisão inteira, do operando à esquerda pelo operando à direita.

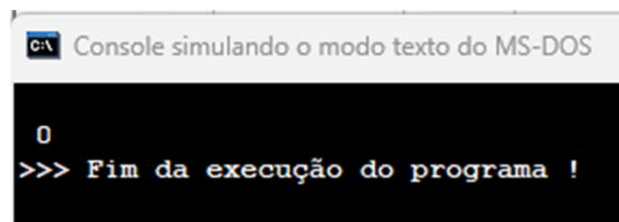
Operadores Aritméticos		
Modulo	%	valor % valor

Por exemplo, $10 / 2 = 5$. 5 é o número inteiro, então tem resto 0. Ele vai comparar o resto dessa divisão apenas. Então, $10 / 2$ gera um resto 0.

```

1 Algoritmo "Operadores%"
2 // Descrição : Aqui vc
3
4 Var
5 // Seção de Declarações
6
7 n1, n2, n3 : real
8 valor : inteiro
9
10 Inicio
11 // Seção de Comandos, pr
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 valor <- n1%2
17 Escreva(valor)
18 Fimalgoritmo

```



FUNÇÕES ARITMÉTICAS

Existem algumas funções que já vêm no Visualg, são elas: para **valor absoluto**, o **outro tipo de exponenciação**, o **valor inteiro**, **raiz quadrada**, π , **seno**, **cosseno**, **tangente** e **graus para radiano**.

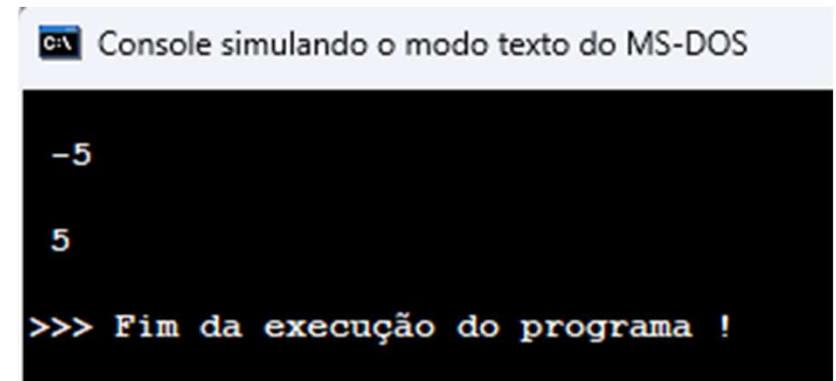
Ao Logo do curso nós veremos como fazer as nossas próprias funções.

Funções aritméticas já existentes

Valor absoluto	Abs	Abs(-10)	10
Exponenciação	Exp	Exp(3,2)	9
Valor inteiro	Int	Int(3.9)	3
Raiz quadrada	RaizQ	RaizQ(25)	5
π	PI	PI	3.14....
Seno	Sen	Sen(0.523)	0.5
Cosseno	Cos	Cos(0.523)	0.86
Tangente	Tan	Tan(0.523)	0.57
Graus para Radiano	GraupRad	GraupRad(30)	0.52

Veja agora alguns exemplos no Visualg, Valor Absoluto:

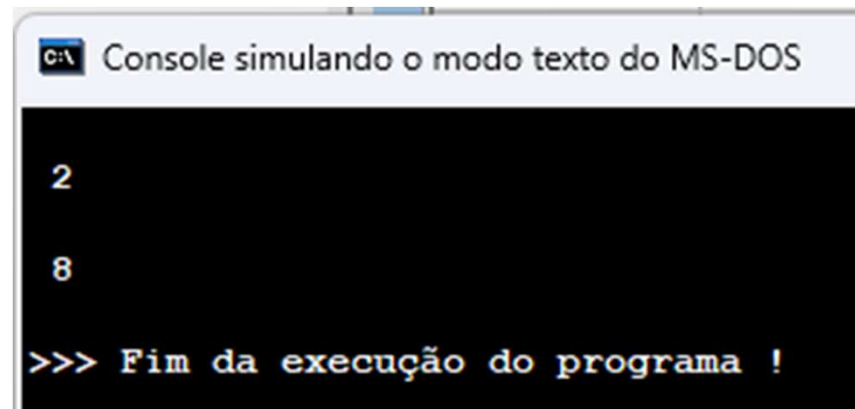
```
1 Algoritmo "valorabsoluto"  
2 // Descrição : Aqui você descreve o que o prog  
3  
4 Var  
5 // Seção de Declarações das variáveis  
6  
7 valor:inteiro  
8  
9 Inicio  
10 // Seção de Comandos, procedimento, funções, ope  
11  
12     valor <- -5  
13     escreval(valor)  
14 //Aqui ele escreve o valor -5  
15     escreval()  
16     valor <-abs(valor)  
17     escreval(valor)  
18 //Aqui ele recebeu o valor absoluto escreve 5  
19 Fimalgoritmo
```



```
C:\> Console simulando o modo texto do MS-DOS  
  
-5  
5  
  
>>> Fim da execução do programa !
```

Veja agora alguns exemplos no Visualg, Exponenciação:

```
1 Algoritmo "Exponenciação"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor: real
8
9 Inicio
10 // Seção de Comandos, procedimentos
11
12     valor <- 2
13     escreval(valor)
14 //Aqui ele escreve o valor 2
15     escreval()
16     valor <- exp(valor,3)
17     escreval(valor)
18 //Aqui ele recebeu a
19 //Exponenciação exibe 8
20 Fimalgoritmo
```



```
C:\> Console simulando o modo texto do MS-DOS

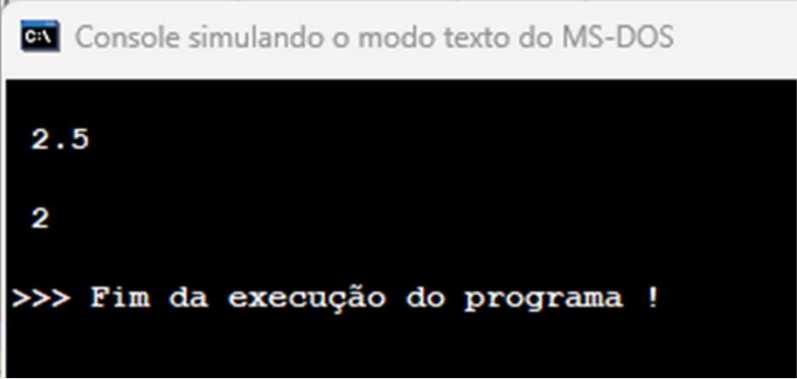
2

8

>>> Fim da execução do programa !
```

Veja agora alguns exemplos no Visualg, Numero Inteiro:

```
1 Algoritmo "Inteiro"
2 // Descrição : Aqui você descreve o
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor: real
8
9 Inicio
10 // Seção de Comandos, procedimento, fu
11
12     valor <- 2.5
13     escreval(valor)
14 //Aqui ele escreve o valor 2.5
15     escreval()
16     valor <- int(valor)
17     escreval(valor)
18 //Aqui ele escreve valor inteiro
19
20 Fimalgoritmo
```



```
C:\ Console simulando o modo texto do MS-DOS

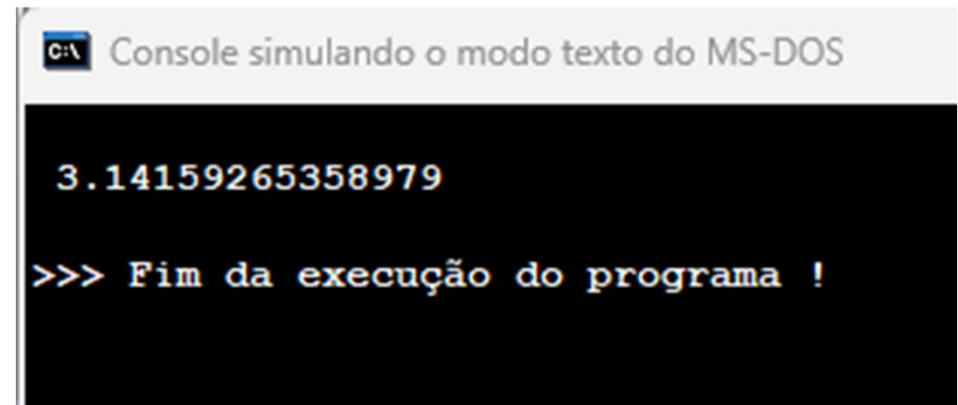
2.5

2

>>> Fim da execução do programa !
```


Veja agora alguns exemplos no Visualg, Valor de π (PI):

```
1 Algoritmo "PI"
2 // Descrição : Aqui você descreve c
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimento, i
11
12     valor <- pi
13     escreval(valor)
14 //Aqui ele escreve
15 //o valor de pi
16 //3.14159265358979
17
18 Fimalgoritmo
```



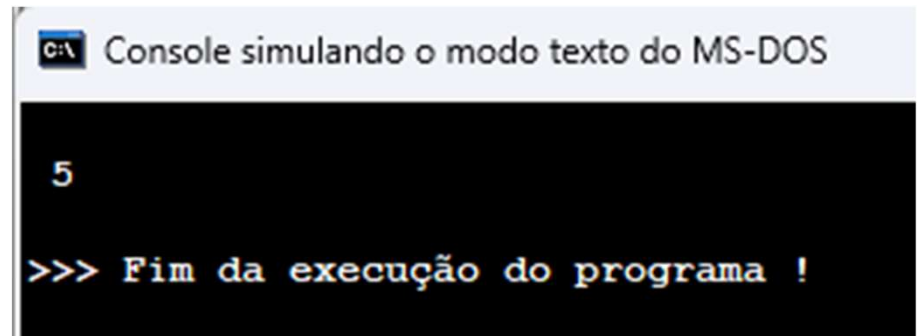
C:\> Console simulando o modo texto do MS-DOS

3.14159265358979

>>> Fim da execução do programa !

Veja agora alguns exemplos no Visualg, Raiz Quadrada:

```
1 Algoritmo "RaizQuadrada"  
2 // Descrição : Aqui você descrev  
3  
4 Var  
5 // Seção de Declarações das variáv  
6  
7 valor:real  
8  
9 Inicio  
10 // Seção de Comandos, procedimento  
11  
12     valor <- raizQ(25)  
13     escreval(valor)  
14 //Aqui ele escreve o valor  
15 //da Raiz Quadrada de 25  
16 // escreve 5  
17  
18 Fimalgoritmo
```



```
C:\> Console simulando o modo texto do MS-DOS  
  
5  
  
>>> Fim da execução do programa !
```

ORDEM DE PRECEDÊNCIA

O que seria a ordem de precedência?

Repare nessa tabela, temos no topo os parênteses, em seguida vem a exponenciação, depois a multiplicação e divisão, e por último, a adição e subtração.

O que significa isso?

Se tenho uma operação grande, por exemplo, $2 + 5 \times 10$, multiplicado por...

Qual será a primeira coisa que será feita?

Isso é regra de matemática, que você já deveria saber. Primeiro, o que é feito são os parênteses, mesmo que seja uma adição, que é a última **ordem de precedência**, será realizado primeiro aquilo que está dentro do **parênteses**. A seguir, ou se não tiver outros **parênteses**, o que será realizado será a **exponenciação**. Depois, a **multiplicação e divisão**, que têm a mesma **ordem de precedência**.

Ordem de Precedência
()
$\wedge$
$\times /$
$+ -$

E, por último, a **adição e multiplicação**. Então, veja um exemplo de **erro** que pode ser gerado caso não prestemos atenção nisso.

Iremos fazer esse valor, receber o $2 + 5 / 2$. Ele responderá $2 + 2.5 = 4.5$

O que eu quero que seja feito? $(5 + 2) = 7$ dividido por $2 = 3,5$.

Isso tem que me gerar um valor 3,5. Isso porque eu não prestei atenção, na **ordem de precedência**.

Vamos mostrar na tela e ver o resultado.

Demonstraremos na tela esse valor. Olha gerou 4,5.

Eu não esperava isso, né?

Então, o que acontece?

Você tem que **prestar atenção na ordem de precedência**.

```
1 Algoritmo "ErroPrecedencia"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimentos
11
12 valor <- 2+5/2
13 escreval(valor)
14 //Aqui ele vai fazer 5/2
15 // depois 2+2.5 = 4.5 ERRADO
16
17 //A equação correta seria
18 //(2+5)/2 = 7/2 = 3.5
19
20 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

4.5

>>> Fim da execução do programa !

Agora veja na forma correta:
 $(5 + 2) = 7$ dividido por 2 = 3,5.

Isso tem que me gerar um
valor 3,5. Agora esta na **ordem
de precedência** correta.

```
1 Algoritmo "ErroPrecedencia"
2 // Descrição : Aqui você descre
3
4 Var
5 // Seção de Declarações das varia
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procediment
11
12     valor <- (2+5)/2
13     escreval(valor)
14
15 //A equação correta seria
16 //(2+5)/2 = 7/2 = 3.5
17
18 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

3.5

>>> Fim da execução do programa !

Mas se eu colocar aqui $2 * 4 / 2$?
Eles têm a mesma ordem de precedência.

Tanto $2 * 4 / 2$, quanto $4 / 2 * 2$, tanto faz a ordem, sempre me trará o mesmo resultado pela questão matemática.

Mas e se eu colocar uma adição aqui depois desse $*$ 2? Então, vou colocar uma soma aqui, um valor 2.

Eu tenho que obrigar esse valor, se eu quiser que ele seja realizado primeiro, colocar ele em parênteses.

Ficando $2 * 4 / (2 + 2)$.

```
1 Algoritmo "Precedencia"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 valor:real
8
9 Inicio
10 // Seção de Comandos, procedimento,
11
12     valor <- 2 * 4 / 2
13     escreval(valor)
14
15 //Tanto faz a ordem,
16 //sempre me trará o mesmo
17 //resultado pela questão matemática
18
19 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

4

>>> Fim da execução do programa !

```
12     valor <- 2 * 4 / (2 + 2)
13     escreval(valor)
14
15 //Para obrigar a soma primeiro
16 //Tenho que colocar entre ( )
17
18 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

2

>>> Fim da execução do programa !

OPERADORES RELACIONAIS

Esses **operadores** são de **comparação**, posso comparar um número com outro, posso comparar uma variável com outra variável, ou uma variável com um número.

Por exemplo, temos aqui o símbolo de maior. Posso comparar **5 > 2**, **verdadeiro** ou **falso**.

É por isso que eles são de comparação, eles são do tipo **lógico**, onde já vimos que tem a variável do tipo **lógico** que traz um **verdadeiro** ou **falso**.

Aqui temos o símbolo **maior**, que é a seta para a direita, **menor**, seta para a esquerda, **maior ou igual**, ou seja, esse número é maior ou é igual a um outro número, **menor ou igual**, temos o **igual**, e o **diferente**.

Esses operadores serão muito utilizados, nas **estruturas condicionais**, nas **estruturas de repetição** entre outras. Precisa ter um domínio sobre eles e a sua utilização.

Ordem de Precedência	
>	Maior
<	Menor
>=	Maior ou igual
=	Menor ou igual
<>	Diferente

Vamos a um exemplo, eu tenho as variáveis **n1**, **n2** e **n3**.

Elas serão do tipo **inteiro**. Iremos criar uma variável **vf** do tipo lógico, **verdadeiro** ou **falso**. Agora adicionar o valor **8** para **n1**, o valor **5** para **n2** e o valor **5** também para **n3**. Precisamos fazer com que **vf** receba **verdadeiro** ou **falso** em uma condição.

Agora iremos perguntar, **n1** é **menor** que **n2**? E vamos mostrar isso na tela. Então, **n1** é 8, **n2** é 5. Então, isso tem que me trazer falso. Vamos **concatenar** com **vf**, que é o resultado, escrevendo: Isso é falso. Isso é maior? Então, 8 realmente é maior que 5. Então, isso é verdadeiro. E **n1 = n2**, **8 = 5** é Falso.

```
1 Algoritmo "OperadoresRelacionais"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 < n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

```
1 Algoritmo "OperadoresRelacionais"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 = n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é FALSO
>>> Fim da execução do programa !
```


Vamos ver o funcionamento do operador diferente, vou perguntar, **n1 é diferente de n2?** n1 é 8, n2 é 5. Eles são diferentes, então isso é verdadeiro. Depois perguntaremos se ele é maior ou igual. **n1 é maior** do que **n2**, 8 é maior do que 5, ele se encaixa na primeira questão, que é maior.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc
3
4 Var
5 // Seção de Declarações das var
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 <> n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você de
3
4 Var
5 // Seção de Declarações das v
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedi
12
13 n1 <- 8
14 n2 <- 5
15 n3 <- 5
16 vf <- n1 >= n2
17 Escreva("Isso é", vf)
18 Fimalgoritmo
```

CA Console simulando o modo texto do MS-DOS

```
Isso é VERDADEIRO
>>> Fim da execução do programa !
```

OPERADORES LÓGICOS

Os **operadores lógicos** são utilizados exclusivamente para a realização de operações entre valores do tipo **lógico**, ou seja, **verdadeiro** ou **falso**.

Muitas vezes é necessário verificar se mais de uma expressão relacional é **verdadeira** para dar prosseguimento ao código.

É nesse ponto que os **operadores lógicos** entram em ação.

São quatro os tipos de **operadores lógicos**, denominados por:

1. **E (AND);**
2. **OU (OR);**
3. **XOU (XOR) e**
4. **NÃO (NOT).**

Iniciando pelo operador **E**, este é utilizado para verificar se um conjunto de valores do tipo lógico são todos **verdadeiros** ou não. Para tanto, é utilizada a seguinte tabela verdade, tendo como base a expressão “**a E b**”, que representa os resultados obtidos.

A E B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	FALSO
FALSO	VERDADEIRO	FALSO
FALSO	FALSO	FALSO

Para que você consiga ler esta tabela, inicialmente, é preciso compreender que está utilizando como base a expressão “**a E b**”, ou seja, quer verificar se o valor correspondente à variável “**a**” e o valor correspondente à variável “**b**” são **verdadeiros**, caso contrário, deverá retornar **falso**.

Pela tabela, é possível observar que apenas caso os dois valores forem **verdadeiros** que o resultado será **verdadeiro**. Toda expressão que utilize o operador **E** segue essa tabela, ou seja, apenas será considerado **verdadeiro** se todos forem **verdadeiros**.

O **operador lógico OU** tem como objetivo verificar se **pelo menos uma das expressões é verdadeira**, ou seja, é preciso que **pelo menos a primeira OU a segunda expressão seja verdade**. Mantendo a expressão anterior como base, apenas alterando o **operador lógico**, você pode analisar a tabela verdade do operador **OU**.

A OU B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	VERDADEIRO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Diferentemente do operador **E**, no caso do operador **OU**, você possui apenas uma possibilidade de a expressão retornar **falsa**, que consiste em **quando todas as operações forem falsas**. Do contrário, o resultado será **verdadeiro**, por ser **preciso pelo menos uma das operações retornando verdade**.

Outro **operador lógico** existente é o operador **XOU**, também conhecido como **OU EXCLUSIVO**. Para esse caso, apenas uma das expressões pode ser **verdadeira**, de forma que ou a primeira ou a segunda expressão deve ser verdade, **caso ambas sejam falsas ou verdadeiras** o resultado é considerado **falso**. A seguir, é apresentada a tabela verdade para o operador **XOU**.

A XOU B		
A	B	RESULTADO
VERDADEIRO	VERDADEIRO	FALSO
VERDADEIRO	FALSO	VERDADEIRO
FALSO	VERDADEIRO	VERDADEIRO
FALSO	FALSO	FALSO

Por fim, existe o operador denominado **NÃO**, também conhecido como **negação**. Esse é o mais simples dos **operadores lógicos**, pois tudo o que faz é **negar um valor lógico**. Ou seja, ao **negar um valor verdadeiro**, você está dizendo que este **valor é falso**. Assim como, ao **negar um valor falso**, entende-se que esse **valor é verdadeiro**.

Pode-se compreender melhor esse operador ao identificar que ele inverte um valor lógico, conforme os exemplos a seguir.

```
a <- nao verdadeiro  
b <- nao falso
```

Nesses casos, as variáveis **a** e **b** são do tipo **lógico**, e no caso da variável **a**, está sendo atribuída a **negação do valor verdadeiro**, ou seja, essa variável receberá o **valor falso**. Referente à variável **b**, esta recebe a **negação do valor falso**, ou seja, o **valor verdadeiro**.

Assim como os operadores aritméticos, os operadores lógicos também possuem uma ordem de prioridade, que consiste no seguinte:

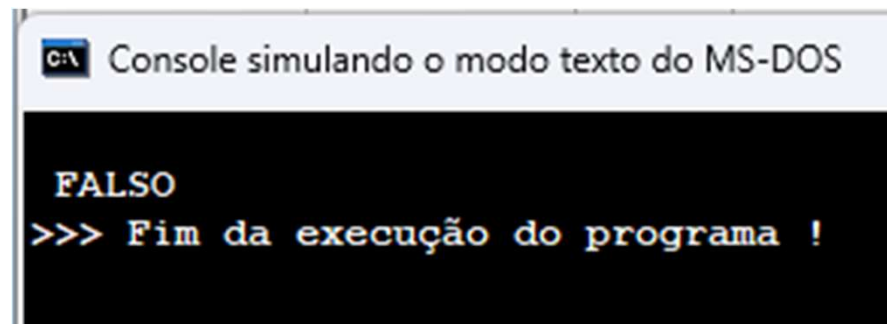
1. NÃO;
2. E;
3. OU.

Lembrando que, caso existam expressões entre parênteses, estas devem ser resolvidas primeiramente, e, caso existam apenas expressões de um único tipo, vamos supor, operador lógico **E**, deve-se iniciar a resolução da esquerda para a direita.

Vamos ver um exemplo agora, no Visualg. Vou digitar aqui, escreva falso e falso. O que dá Falso com Falso?

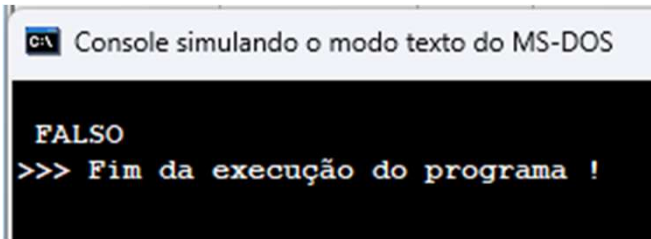
Eu **não tenho o bolo(Falso)** e **não tenho o refrigerante(Falso)**, então, eu **não levo**. Então, **Falso com Falso** é igual a **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui voce
3
4 Var
5 // Seção de Declarações de
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, proc
12
13 Escreva(falso e falso)
14 Fimalgoritmo
```



Verdadeiro e Falso, eu sou aquela pessoa fresca, eu quero os dois. Se **eu tenho o bolo(Verdadeiro)**, mas **não tenho o refrigerante(Falso)**, eu também **não vou levar(Falso)**. Isso vai me gerar **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc.
3
4 Var
5 // Seção de Declarações das var.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime.
12
13 Escreva(verdadeiro e falso)
14 Fimalgoritmo
```



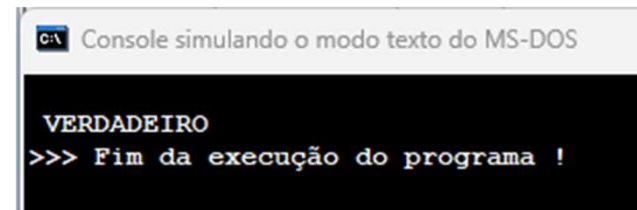
Console simulando o modo texto do MS-DOS

FALSO

>>> Fim da execução do programa !

Agora, se eu colocar **Verdadeiro** e **Verdadeiro**, ou seja, eu tenho o **bolo(Verdadeiro)** e tenho o **refrigerante(Verdadeiro)**, então eu levo **verdadeiro**. Agora eu vou levar(**Verdadeiro**). Isso vai me gerar um **Verdadeiro**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 Escreva(verdadeiro e verdadeiro)
14 Fimalgoritmo
```

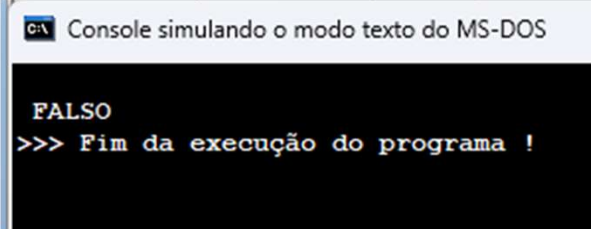


Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Vamos ver a utilização do **OU**. **Falso ou Falso**. Eu não tenho nenhum dos dois, então, **eu não vou levar**. Isso também vai me gerar um **Falso**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você desc
3
4 Var
5 // Seção de Declarações das var
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedime
12
13 Escreva(falso ou falso)
14 Fimalgoritmo
```



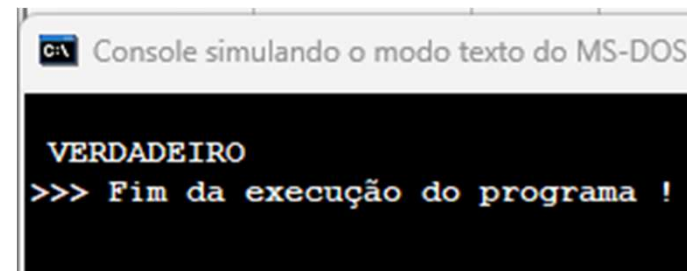
C:\ Console simulando o modo texto do MS-DOS

FALSO

>>> Fim da execução do programa !

Verdadeiro ou Verdadeiro. Então, eu tenho as duas coisas e eu vou levar também. Isso também vai me gerar um **Verdadeiro**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis:
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 Escreva(verdadeiro ou verdadeiro)
14 Fimalgoritmo
```

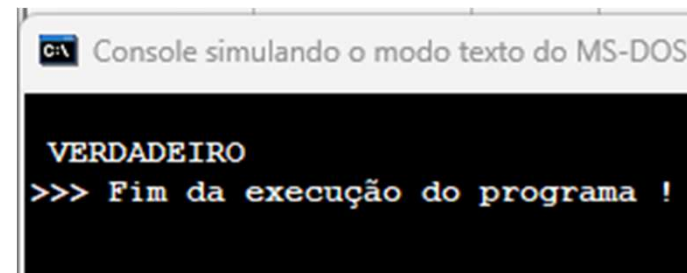


C:\ Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Verdadeiro ou Falso. OU é uma coisa ou outra, eu posso levar também um dos dois. Então, se eu só tenho o bolo, eu vou levar o bolo. Então, isso é Verdadeiro.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimentos
12
13 Escreva(verdadeiro ou Falso)
14 Fimalgoritmo
```

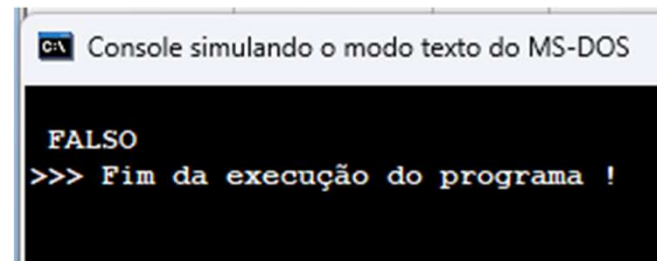


C:\ Console simulando o modo texto do MS-DOS

VERDADEIRO
>>> Fim da execução do programa !

Agora, vamos **negar** tudo isso. **Coloco em parênteses não**, ser negado. Se era **Verdadeiro**, agora é **Falso**.

```
1 Algoritmo "Operadores"  
2 // Descrição : Aqui você descreve  
3  
4 Var  
5 // Seção de Declarações das variáveis  
6  
7 n1, n2, n3 : real  
8 vf : logico  
9  
10 Inicio  
11 // Seção de Comandos, procedimento,  
12  
13 Escreva(na|(verdadeiro ou Falso))  
14 Fimalgoritmo
```



Console simulando o modo texto do MS-DOS

```
FALSO  
>>> Fim da execução do programa !
```

Vou complicar um pouco agora. Vou fazer o **N1** receber **10**, **N2** receber **5**, e **N3** receber **75**. Escreva o seguinte. **N1** é maior do que **N2**? Então, isso é uma **Verdade**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você
3
4 Var
5 // Seção de Declarações da
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, proc
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva(n1 > n2)
18 Fimalgoritmo
```

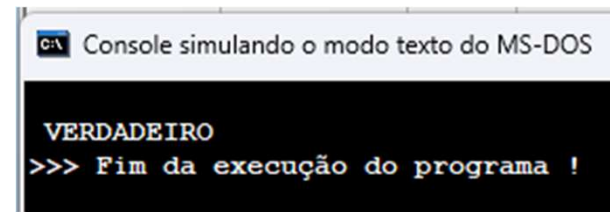
C:\ Console simulando o modo texto do MS-DOS

```
VERDADEIRO
>>> Fim da execução do programa !
```

Vamos colocar tudo isso em parênteses e vou fazer uma comparação do tipo **E**. E eu coloco o **Verdadeiro**. Perceba que estou fazendo a mesma coisa que antes.

Estou perguntando se **Verdadeiro** e **Verdadeiro** vai dar o quê? Se **tenho o bolo(Verdadeiro)** e **tenho o refrigerante(Verdadeiro)**, as duas coisas são **Verdades**.

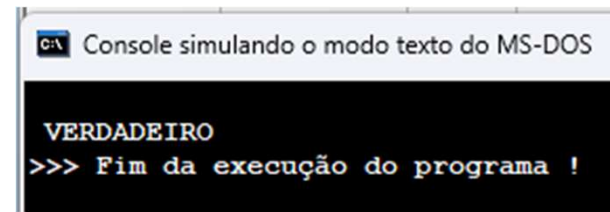
```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva((n1 > n2) e verdadeiro)
18 //Verdadeiro e Verdadeiro
19 Fimalgoritmo
```



Vamos colocar tudo isso em parênteses e vou fazer uma comparação do tipo **E**. E eu coloco o **Verdadeiro**. Perceba que estou fazendo a mesma coisa que antes.

Estou perguntando se **Verdadeiro** e **Verdadeiro** vai dar o quê? Se **tenho o bolo(Verdadeiro)** e **tenho o refrigerante(Verdadeiro)**, as duas coisas são **Verdades**.

```
1 Algoritmo "Operadores"
2 // Descrição : Aqui você descreve
3
4 Var
5 // Seção de Declarações das variáveis.
6
7 n1, n2, n3 : real
8 vf : logico
9
10 Inicio
11 // Seção de Comandos, procedimento,
12
13 n1 <- 10
14 n2 <- 5
15 n3 <- 75
16
17 Escreva((n1 > n2) e verdadeiro)
18 //Verdadeiro e Verdadeiro
19 Fimalgoritmo
```



ESTRUTURAS DE CONTROLE OU ESTRUTURAS DE DECISÃO

Conhecidos como **SE** ou o famoso **IF**.

Para que servem as estruturas de controle?

Estas estruturas são fundamentais, porque praticamente todo sistema que você vai desenvolver, irá utilizar elas.

É uma estrutura que você vai definir o que o seu programa vai fazer caso uma coisa seja verdade, caso uma coisa não corresponda àquilo que você quer.

Entenderemos isso melhor com um exemplo.

Vamos supor aqui:

**SE EU RECEBER HOJE ENTÃO
EU VOU AO MERCADO**

É uma decisão que eu quero tomar. Estou dizendo, **se eu receber hoje, se isso for uma verdade, se isso acontecer, então eu vou ao mercado**

SENÃO

VOU FICAR EM CASA

FIMSE

Caso contrário, ou seja, **se não, se eu não receber**, então eu vou ficar em casa. Veja que tenho duas opções onde o meu sistema pode realizar.

Por exemplo, **se** eu tenho um número que a pessoa digitou lá no teclado, e esse número for maior que 10, eu imprimo na tela: Seja bem-vindo. **Se não**, quando eu insiro **se não**, eu estou dizendo que qualquer outro número que não seja o 10, for enviado, **então** ele não vai entrar no sistema, não vai, por exemplo, imprimir uma mensagem de Seja bem-vindo.

Essa é a lógica, que é utilizaremos. Você precisa utilizar duas palavras. **Então**, é nessa linha aqui, **se eu receber hoje**, você vai escrever **então**, aí eu faço alguma coisa. E lá no final, você coloca um **fimse**, você está delimitando, que é aí que acaba a sua estrutura.

Outro exemplo, vamos imaginar uma situação fictícia, passará um filme de terror no cinema e a pessoa só pode entrar se ela tiver **mais de 18 anos ou 18 anos**. Isso é uma **condição** também, **então se a pessoa for maior ou igual a 18 anos, tiver mais de 18 anos, ela pode entrar para assistir esse filme.**

SE PESSOA $\geq$ 18 ANOS ENTÃO
PODE ENTRAR
SENÃO
ENTRADA PROIBIDA
FIMSE

Caso contrário, **ela está proibida**. Entenda que quando falo **se não**, eu digo que todas as outras opções que não sejam igual a essa, **não poderá realizar**, entrar nessa parte de cima do **se**, ele vai direto para o **se não**.

Perceba também que estou colocando de forma bem organizada, linha embaixo de linha, então o **se**, veja que ele corresponde ao **se não**, e for colocar o **fimse**, ele também vai seguir a mesma linha.

E sempre que eu colocar alguma coisa dentro dessa estrutura, eu dou um **tab**, um espaço, para ficar visível.

Isso você vai utilizar sempre, se chama **indentação**. Um bom programador sabe utilizar a **indentação** de forma que o sistema fique bem estruturado, não fica desorganizado.

Temos duas estruturas, a **simples** e a **composta**. A **simples**, ela não contém aquela palavra **se não**.

Então, o que acontece?

Se a pessoa for maior ou igual a 18, pode entrar. E o programa imprime **Pode Entrar**.

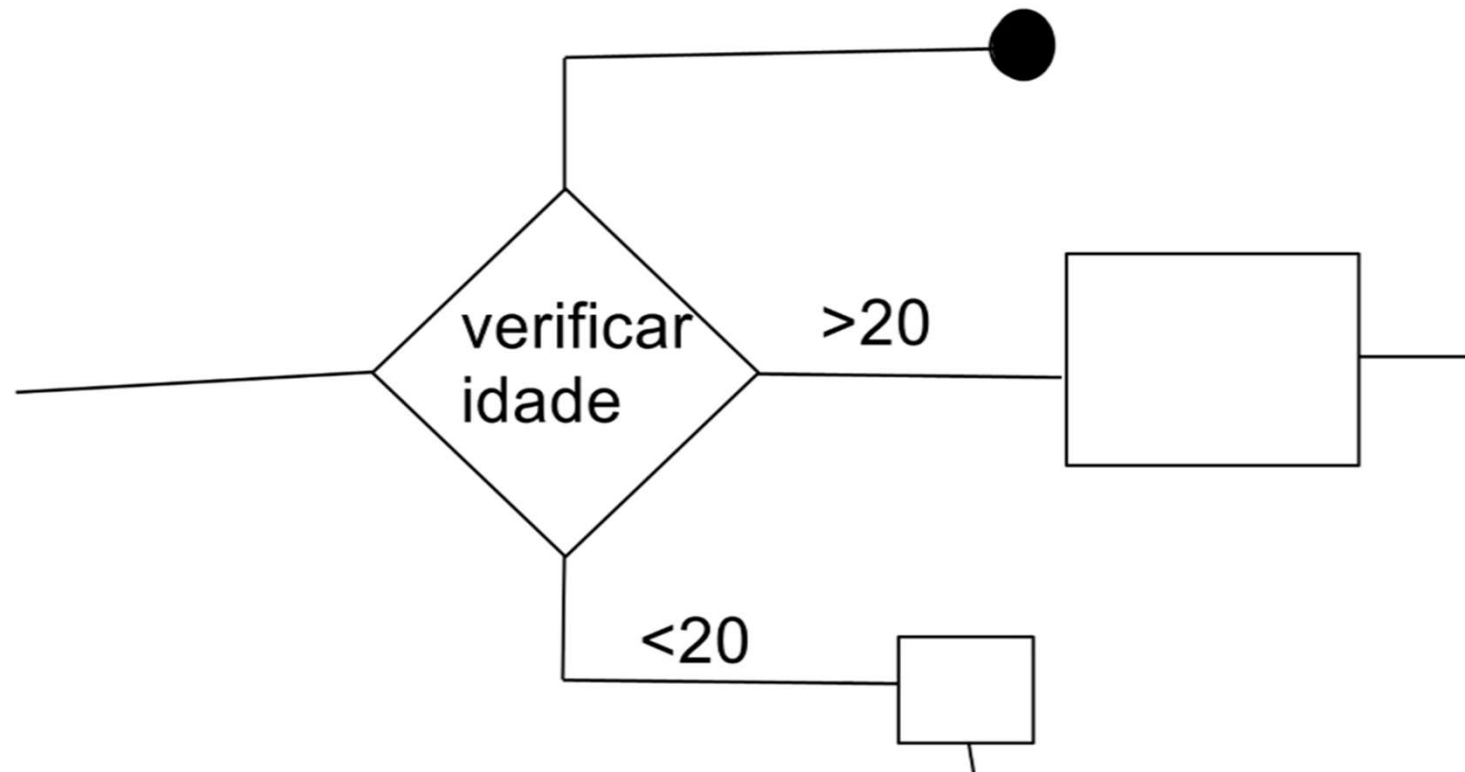
Se a pessoa tiver menos de 18, o programa não vai fazer nada, não vai escrever não pode entrar, não vai fazer nada. Só vai escrever pode entrar **caso tenha mais de 18 anos**.

Essa é uma **estrutura simples** que não tem o **se não**.

A **estrutura composta**, podemos colocar o **se não**, e também podemos colocar uma **estrutura condicional aninhada** (quando uma estrutura de decisão está localizada dentro do lado falso da outra), que é uma estrutura com vários **se não**.

Se a pessoa for menor que 5 anos, então ela é uma criança.

Se não, se uma pessoa tiver até 18 anos, adolescente. Se não, então eu posso colocar várias condições,
Veja aqui como seria uma estrutura no fluxograma.



Temos um caminho de código, e quando eu chego na estrutura condicional, eu coloco esse símbolo, e eu escrevo alguma coisa dentro desse símbolo.

Por exemplo, verificar idade. **Se a idade for maior que 20**, faço uma coisa. **Se for menor que 20**, faço outra, e tenho também o caminho que vai para o final desse fluxograma.

Sempre que tiver a bolinha, significa que chegou no final da estrutura, no final do programa.

Toda vez que for utilizar a **estrutura condicional** no fluxograma, será assim. Com esse símbolo, você coloca o que você quer fazer dentro, e coloca as setas que vão mostrar o caminho.

Agora Vamos Praticar alguns exercícios para fixar esse conteúdo.

1º Verificar qual numero é maior

Para verificar se um número é menor ou se é maior, vamos pedir para que o usuário entre com esta informação.

Então, vamos escrever, Escreval, digite um número.

Agora leia N1 para armazenar aquilo que for digitado em N1, e vamos criar essa variável N1, que será do tipo inteiro. A pessoa precisa digitar um número inteiro.

Vamos criar o N2 para armazenar o segundo número. Escreva Digite outro número.

Após digitar esses dois números, precisamos verificar qual é o número maior. Agora faremos o seguinte.

Precisamos colocar o fim se, se N1 for maior que N2, vamos dizer que N1 é maior que N2. Escreva o número, aí concatenamos com N1, é maior que N2. O número N1 é maior que o número 2. Caso contrário, se N1, se esse primeiro número não for maior, ele só pode ser menor, existe outra possibilidade, que é o número ser igual, mas não iremos abordar isso aqui.

Digite um número, vou colocar o número 10, digite outro número, eu vou colocar o número 25. Se N1 for maior que N2, então o número 1, N1 será maior que N2, mas caso contrário, ele é menor.

Veja o resultado: o número 25 é maior do que 10.

```
1 Algoritmo "semnome"
2 // Descrição : Aqui você descreve o que o programa faz
3
4 Var
5 // Seção de Declarações das variáveis
6 n1, n2 : inteiro
7
8 Inicio
9 // Seção de Comandos, procedimento, funções, operações
10 escreval("Digite um numero")
11 leia(n1)
12 escreval("Digite outro numero")
13 leia(n2)
14 se(n1 > n2) entao
15     escreval("O numero ",n1," é maior que ",n2)
16 senao
17     escreval("O numero ",n2," é maior que ",n1)
18 fimse
19
20 Fimalgoritmo
```

C:\ Console simulando o modo texto do MS-DOS

```
Digite um numero
10
Digite outro numero
25
O numero 25 é maior que 10

>>> Fim da execução do programa !
```

Vejam agora o próximo exercício:

Nesse exercício, queremos verificar se o número é par ou ímpar.

Para isso, nós temos que pensar, como nós sabemos se o número é par ou ímpar?

Sabemos que os números pares, eles são terminados em 0, 2, 4, 6, 8, ...

Na matemática, se você divide qualquer número pelo primeiro número par, o primeiro número par é o 2, você terá resto 0. Se você tiver algum resto que não seja 0, significa que esse número é ímpar.

Veja o exemplo, 30 dividido por 2 = 15, ou seja, 2 vezes 15 dá 30, que tirando, subtraindo 30 com 30 dá 0, que é o resto. Então, você percebe que o número 30 é um número par.

$$\begin{array}{r|l} 30 & 2 \\ - 30 & \\ \hline 0 & 15 \end{array}$$

Veja agora com o número ímpar.

Tenho 7, que é ímpar, se eu tentar dividir por 2, ele me retornará 3. faremos 2 vezes 3, vai dar 6, iremos subtrair, teremos resto 1, ou seja, sempre que retornar algum resto que não seja 0, será um número ímpar.

$$\begin{array}{r|l} 7 & 2 \\ - 6 & 3 \\ \hline 1 & \end{array}$$


E agora como representar isso dentro do algoritmo?

Nós utilizaremos o módulo.

Primeiro o usuário deve entrar com uma variável, com um valor, iremos chamar de **n**, será do tipo inteiro para a verificação se é **par** ou **ímpar**. **Se n, % “módulo” 2 for igual a 0**, ele vai pegar esse número, qualquer número que seja, dividir por 2 e verificar o resto.

```

1 Algoritmo "exercicios"
2
3 Var
4 // Seção de Declarações das variáveis
5 n : inteiro
6
7 Inicio
8 // Seção de Comandos, procedimento, fun
9 escreval("Digite um numero: ")
10 leia(n)
11     se (n%2 = 0) entao
12         escreva("O numero", n, " é par")
13
14         senao
15             escreva("O numero", n, " é impar")
16     fimse
17 Fimalgoritmo

```

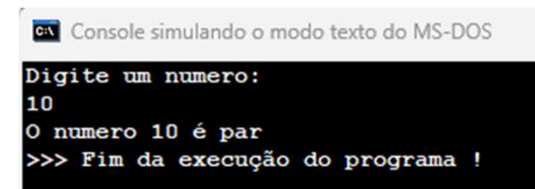
Digite o número 10 que é par.

Vamos o número 13 que é ímpar.

Se o resto for igual a 0, ele é um número par, então, escreva, o número n é par.

Caso contrário, se ele não é par, ele só pode ser ímpar, não existe outra possibilidade.

Então, escreva, caso contrário, o número n é ímpar, e fimse.

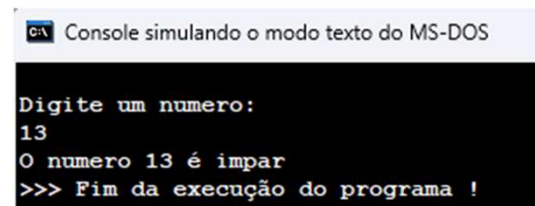


```

C:\> Console simulando o modo texto do MS-DOS

Digite um numero:
10
O numero 10 é par
>>> Fim da execução do programa !

```



```

C:\> Console simulando o modo texto do MS-DOS

Digite um numero:
13
O numero 13 é impar
>>> Fim da execução do programa !

```

Próximo exercício: Crie um sistema que verifique a senha e imprima uma mensagem de erro ou sucesso, acessível somente com a senha 1234.

Primeiro passo é criar a variável senha, é do tipo inteiro. Essa senha receberá o valor 1234. E iremos pedir ao usuário que digite a senha.

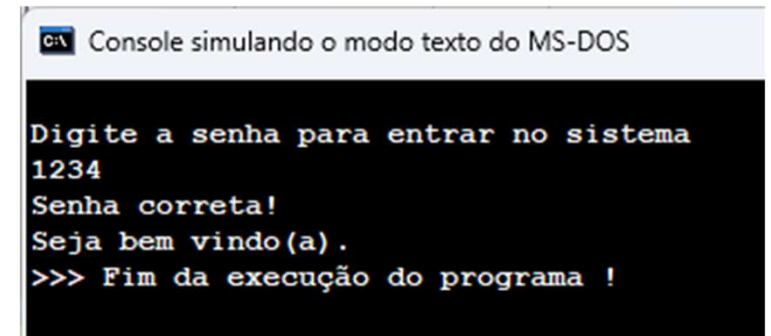
Assim, escreva, digite a senha para entrar no sistema. Agora preciso ler essa senha e armazenar em uma variável chamada de senhaDigitada, também do tipo inteiro.

```
1 Algoritmo "senha"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5 senha, senhaDigitada : inteiro  
6  
7 Inicio  
8 // Seção de Comandos, procedimento,  
9 senha <- 1234
```

Se a senhaDigitada for igual à senha, mostrar a mensagem que a pessoa está autorizada para acessar o sistema.

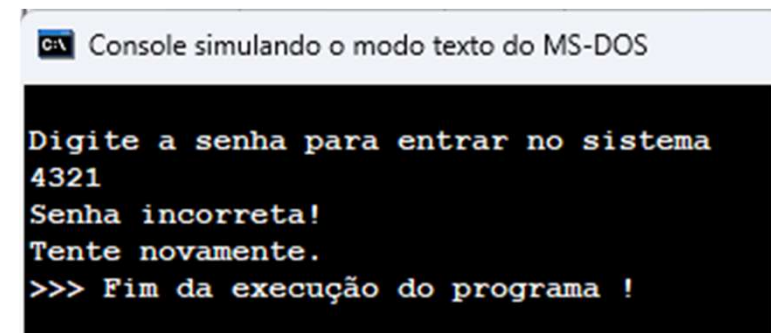
Então, escreval, **senha correta**, e mostre uma mensagem de boas-vindas, **Seja bem-vindo(a)!**. Se não, **Senha está incorreta!**, **Tente novamente**. E o fimse.

```
11 escreval("Digite a senha para entrar no sistema")
12 leia(senhaDigitada)
13     se(senhaDigitada = senha) entao
14         escreval("Senha correta!")
15         escreva("Seja bem vindo(a).")
16     senao
17         escreval("Senha incorreta!")
18         escreva("Tente novamente.")
19     fimse
20 Fimalgoritmo
```

A screenshot of a DOS console window titled "C:\ Console simulando o modo texto do MS-DOS". The window has a black background with white text. The text displayed is: "Digite a senha para entrar no sistema", "1234", "Senha correta!", "Seja bem vindo(a).", and ">>> Fim da execução do programa !".

```
C:\ Console simulando o modo texto do MS-DOS

Digite a senha para entrar no sistema
1234
Senha correta!
Seja bem vindo(a).
>>> Fim da execução do programa !
```

A screenshot of a DOS console window titled "C:\ Console simulando o modo texto do MS-DOS". The window has a black background with white text. The text displayed is: "Digite a senha para entrar no sistema", "4321", "Senha incorreta!", "Tente novamente.", and ">>> Fim da execução do programa !".

```
C:\ Console simulando o modo texto do MS-DOS

Digite a senha para entrar no sistema
4321
Senha incorreta!
Tente novamente.
>>> Fim da execução do programa !
```

EXERCÍCIO

Crie um algoritmo de uma calculadora simples:

Para iniciar precisamos fazer basicamente três coisas:

A primeira delas é pedir dois números, mesmo que a pessoa realize uma multiplicação, uma adição, uma divisão ou uma subtração, ela sempre precisará enviar dois números que serão armazenados em variáveis.

A segunda é pedir qual é a operação, a pessoa envia os dois números e a operação que ela deseja realizar.

A terceira é evitar erros, veremos que, como programadores, precisamos evitar o máximo de erros possível para que nosso software funcione.

Mesmo que seja apenas algoritmos e não uma linguagem de programação. Mas já é bom ver como é o funcionamento, como precisa pensar para evitar o máximo de erros possíveis.

EXERCÍCIOS

```
1 Algoritmo "calculadora"
2
3 Var
4 // Seção de Declarações das variáveis
5
6 n1, n2, res : real
7 operacao : inteiro
8 Inicio
9 // Seção de Comandos, procedimento, funções
10 escreva("Digite um número: ")
11 leia(n1)
12 escreva("Digite outro número: ")
13 leia(n2)
14 escreval()
15 escreval("Digite 1 para adição")
16 escreval("Digite 2 para subtração")
17 escreval("Digite 3 para multiplicação")
18 escreval("Digite 4 para divisão")
19 escreval
20 leia(operacao)
```

A primeira coisa será pedir dois números. Vamos escrever, digite um número e leia esse número, guardar em n1, digitar outro número e guardaremos em n2, criar as variáveis correspondentes do tipo real e pedir a operação.

Dar as opções, digite 1 para a adição, 2 para a subtração, 3 para a multiplicação e 4 a divisão, ler essa operação e criar a sua variável correspondente.

Como as opções são um, dois, três ou quatro, então nós criaremos do tipo inteiro, verificar qual número foi digitado, se a operação for igual a 1, criar uma variável para guardar a resposta, res do tipo real


```

20 leia(operacao)
21   se(operacao = 1) entao
22     res <- n1 + n2
23   senao
24     se(operacao = 2) entao
25       res <- n1 - n2
26     senao
27       se(operacao = 3) entao
28         res <- n1 * n2
29       senao
30         se(operacao = 4) entao
31           se(n2 = 0) entao //tratar divisao por 0
32             escreval("Não é possível dividir por 0")
33           senao
34             res <- n1 / n2
35           fimse
36         senao // aqui tem que responder caso a operação
37           // seja fora das 4 opções
38           escreval("Operação Invalida")
39         fimse
40       fimse
41     fimse
42   fimse
43 escreva ("O resultado é:", res)
44 Fimalgoritmo

```

Se a operação for igual a 1, res vai receber, a opção 1 é a adição, então $n1 + n2$, acrescentar um se se não se, a operação for igual a 2, será feita uma subtração. Se não, fim se, se a operação for igual a 3, se a operação for igual a 4, divisão.

E agora, nesse último se não, nós precisamos pensar na questão de a pessoa digitar um número que não seja de 1 a 4.

Ou seja, será uma operação inválida. Iremos mostrar na tela uma mensagem dizendo que a operação é inválida.

Agora nós evitamos o erro, a divisão com 0, é o segundo número que não pode ser 0, dentro aqui da opção 4, se $n2$, for igual a 0, não posso fazer essa operação, escreval Não é possível dividir por 0, se não, aí sim eu posso fazer o cálculo.

ESTRUTURA CONDICIONAL ESCOLHA, ESCOLHA CASO

Para que ela serve?

Nas linguagens de programação você encontrará como **switch**, **switch case**, que é o **escolha** e **escolha caso**.

Ela é uma estrutura também condicional, o funcionamento é muito parecido com o **SE**, mas é um pouco limitada, nós não podemos fazer comparação, por exemplo, que nem nós fazíamos no **SE**.

Se o número for menor que 2,5, se o número for igual a 2 e o outro for entre um número e outro, ela não consegue fazer isso, esta condicional é mais para números específicos.

Se o número for 1, 2 ou 3, ou se esse caractere for, por exemplo, de adição, de multiplicação, é pra isso que nós utilizamos pra um sistema mais básico, assim, menos complexo, e a estrutura não fica aninhada, igual o **SE**, que ficava um **SE** dentro do outro e formando uma montanha de código, aqui fica mais curto, porém ele também é um pouco mais limitado.

Veja um exemplo:

Crie uma variável valor do tipo inteiro, faça essa variável receber o número 2. Depois escreveremos **escolha**, o nome da variável que nós queremos utilizar, nesse caso a variável valor que contém o número 2, e colocar a palavra **caso**.

Caso 1, que significa caso 1? Caso o valor que está dentro dessa variável valor seja 1, eu vou escrever **OI**.

Caso 2, é o que esperamos, então escreva **TCHAU**.

Outro caso, ou seja, qualquer outro número, qualquer outra opção, iremos escrever **OI e TCHAU**, no fim, vem o **fim escolha**.

VALOR : INTEIRO

VALOR <- 2

ESCOLHA VALOR

CASO 1

ESCREVA("OI")

CASO 2

ESCREVA("TCHAU")

OUTROCASO

ESCREVA("OI E TCHAU")

FIMESCOLHA

EXERCÍCIO

Faremos uma calculadora também para ver como que é a diferença de uma calculadora usando o **se** e usando o **escolha caso**.

A ideia é a mesma, então vamos criar as variáveis n1 e n2 e a operação. Só que a operação não será do tipo inteiro, será do tipo caractere, para que a pessoa digite os mesmos símbolos das operações e o resultado do tipo real.

```
1 Algoritmo "calculadora"  
2  
3 Var  
4 // Seção de Declarações das variáveis  
5  
6 n1, n2, res : real  
7 operacao : caractere
```

EXERCÍCIOS

```
8 Inicio
9 // Seção de Comandos, procedimento, funções,
10 escreva("Digite um número: ")
11 leia(n1)
12 escreva("Digite outro número: ")
13 leia(n2)
14 escreval("Qual operação deseja realizar?")
15 escreval("+ - para somar")
16 escreval("- - para subtrair")
17 escreval("x - para multiplicar")
18 escreval("/ - para dividir")
19 leia(operacao)
20 escreval()
21   escolha operacao
22   caso "+"
23       res <- n1 + n2
24   caso "-"
25       res <- n1 - n2
26   caso "x"
27       res <- n1 * n2
28   caso "/"
29       res <- n1 / n2
30   outrocaso
31       escreval("Operação Invalida")
32   fimescolha
33 escreva ("O resultado é:", res)
34 Fimalgoritmo
```

Iremos iniciar pedindo os números que serão calculados. Digite o primeiro número. Leia n1, Digite o segundo número, leia n2, e escrever qual operação deseja realizar, exibir as opções com o símbolo de + para somar, - para subtrair, um x para multiplicar e uma / para dividir

Agora ler a operação, escrever escolha operação, caso o símbolo seja +, somar e guardar na variável res, caso seja o símbolo de - subtração.

Fazer o mesmo para a multiplicação e a divisão, outro caso, seja este um outro símbolo, mostrar uma mensagem, operação inválida e mostrar o resultado.